Nexus HMS Non-Functional Requirements

Nexus HMS Enterprise Edition

Document control

- Product: Nexus HMS Enterprise Hotel Management System
- Release baseline: Phase 7B Enterprise Platform
- Document type: Non-functional requirements specification
- Date: 27 August 2026

1. Quality objectives

Nexus HMS shall be secure by default, operationally observable, responsive for normal hotel workloads, maintainable by a TypeScript team, and deployable from a repeatable build. Requirements in this document are targets and operating expectations. They should be measured in each production environment rather than assumed from development results.

2. Performance and capacity

NFR-001 Response time

- Health checks should complete in under 250 ms when the database is healthy.
- Normal authenticated list and detail requests should complete within 1 second at the 95th percentile under the agreed baseline load.
- Dashboard and BI requests should complete within 2 seconds at the 95th percentile for a single property and normal seeded data volume.
- API requests that exceed the target should be visible through request-duration metrics.

NFR-002 Throughput and throttling

- The default request throttle is 300 requests per minute per throttling key.
- Production shall use Redis 7 with the Socket.IO Redis adapter so realtime packets can reach clients connected to different API replicas.
- The Kubernetes baseline shall retain three API replicas and an HPA range of two to ten replicas.
- Load tests should cover authentication, reservations, dashboard, search, reports, and mobile synchronization separately.

NFR-003 Caching

- Expensive dashboard and reservation reads may use a 60-second in-memory cache.
- Cache entries shall have explicit expiry and invalidation behavior.
- Cache flush is an administrative operation and shall be observable in audit logs where enabled.
- The in-memory cache is process-local and shall not be described or operated as Redis-backed. Cross-instance cache consistency is not provided.

3. Availability and resilience

NFR-004 Health and startup

- The API shall expose ‘/api/health‘ with a machine-readable status and timestamp.
- A deployment shall use health checks for process readiness and liveness.
- The application shall fail startup when the required MFA encryption secret is absent; it shall not silently fall back to a weak secret for MFA data.
- The production API shall fail startup when ‘REDIS_URL‘ is absent so cross-replica Socket.IO delivery is not silently disabled.
- Redis readiness checks and application connections shall authenticate; an unauthenticated Redis URL shall not be used.

NFR-005 Failure behavior

- Invalid credentials, invalid tokens, invalid TOTP codes, forbidden administrative actions, malformed input, and missing resources shall return explicit HTTP errors.
- A failed security check shall not silently grant access.
- A malformed stored IP rule shall be ignored safely and should be investigated through operational review.
- A failed mobile synchronization request shall not advance the client’s sync watermark.

NFR-006 Recovery

- PostgreSQL backups shall be scheduled according to the hotel’s recovery point objective.
- Restore procedures shall be rehearsed in a non-production environment.
- Deployments shall retain a previous application image or build so that rollback is possible without rebuilding from an unverified commit.
- The Kubernetes 10Gi ReadWriteOnce uploads PVC shall use required API pod affinity to co-locate API replicas on one node. Multi-node placement shall not be enabled until shared filesystem or object storage replaces that boundary.

4. Security and privacy

NFR-007 Authentication and authorization

- JWT access tokens shall be required for protected routes.
- MFA-pending tokens shall be cryptographically distinct from full access tokens and shall have a short lifetime.
- Refresh tokens shall use a separate signing secret derived from the session secret.
- Role and tenant scope shall be evaluated server-side; client-side navigation is not an authorization boundary.

NFR-008 Secret and credential protection

- ‘SESSION_SECRET‘ shall be supplied through the environment secret store.
- ‘REDIS_PASSWORD‘ and the credential-bearing ‘REDIS_URL‘ shall be supplied through the environment secret store and shall contain matching credentials.
- Real production should use managed Redis on a private network with TLS and a ‘rediss://‘ URL. The included internal Redis service shall not be represented as TLS-enabled.
- MFA TOTP secrets and backup codes shall be encrypted at rest with AES-256-GCM using a key derived from ‘SESSION_SECRET‘.
- Backup codes shall be one-time use.
- Secrets, access tokens, refresh tokens, QR payloads, and backup codes shall not be written to logs.
- Production systems shall not use demo credentials or a default session secret.

NFR-009 Network protection

- Helmet shall be enabled for standard HTTP security headers.
- CORS shall be configured to the approved application origins in production rather than left broadly permissive.
- The Prometheus metrics endpoint is public by design for scrape compatibility and shall be restricted at the ingress, firewall, or network-policy layer.
- IP restriction rules shall accept only validated IPv4 addresses or IPv4 CIDR notation.
- Kubernetes NetworkPolicies shall allow Redis ingress only from API pods on TCP 6379 and deny all Redis egress.
- Kubernetes Redis and web containers shall run as non-root, disallow privilege escalation, and drop all Linux capabilities.

NFR-010 Data isolation

- Tenant-aware services shall filter by authenticated tenant context.
- No service shall return cross-tenant core hotel rows while a safe tenant_id filter is unavailable.
- A future core-table tenant migration shall include indexes, backfill, constraints, and a verification plan before tenant operational sync is enabled.

5. Observability

NFR-011 Metrics

The metrics endpoint shall expose Prometheus-compatible text for:

- Total HTTP requests.
- Requests by method and route.
- Average request duration and duration count.
- Active database pool connections where available.
- Active WebSocket connections.
- Cache hits and misses.

NFR-012 Logs and audit

- Application logs shall capture startup, fatal errors, and operational warnings without secret values.
- Audit records should include actor, action, resource, method, status, and relevant request metadata.
- Security administrators shall be able to review active devices, IP rules, and audit activity.

NFR-013 Alerting

Operations should alert on failed health checks, elevated 5xx rates, high request latency, database exhaustion, repeated authentication failures, WebSocket disconnect spikes, and backup failures.

6. Accessibility and localization

NFR-014 Accessibility

- Interactive controls shall have accessible names and keyboard focus behavior.
- Color shall not be the sole indicator of room, reservation, or task state.
- Forms shall expose validation errors adjacent to the relevant field.
- Charts shall have a textual summary or data table path for users who cannot interpret a visual chart.

NFR-015 Localization

- The user interface shall support English, Amharic, Oromo, Arabic, and French.
- Locale selection shall persist across sessions.
- Arabic shall apply right-to-left document direction.
- Dates, numbers, currency, and time zones shall be rendered with a documented property-level convention.

7. Maintainability and delivery

NFR-016 Code quality

- Backend features shall follow the module, controller, service, and module-registration pattern.
- Shared database schema changes shall be compiled before the API build consumes them.
- API changes should update the OpenAPI contract or relevant client types.
- Security-sensitive behavior shall have durable integration coverage for MFA, authorization, IP rules, and token refresh.

NFR-017 Build reproducibility

- The repository shall use pnpm workspaces and a committed lockfile.
- CI should run type checking, builds, tests, and image validation before release.
- Container validation shall verify PostgreSQL readiness, authenticated Redis readiness, production health for two API replicas, and non-root write access to the shared upload volume.
- The API and web applications shall be independently buildable from the workspace.

NFR-018 Compatibility

- Supported browsers shall include current Chromium, Firefox, and Safari releases used by hotel workstations.
- The API shall use JSON request and response payloads except for multipart document uploads and the Prometheus text endpoint.
- WebSocket clients shall use Socket.IO-compatible transports and authentication.
- Cross-replica Socket.IO delivery shall use ‘@socket.io/redis-adapter‘ with the Redis client and Redis 7.

8. Operational constraints and trade-offs

- Cache is process-local rather than Redis-backed.
- Event delivery uses an in-process EventEmitter2 bus rather than RabbitMQ or Kafka.
- Redis 7 is used only as the Socket.IO cross-replica backplane; it is not a shared cache or durable EventBus.
- Included Compose and Kubernetes Redis use password authentication without TLS; managed/private TLS Redis is the recommendation for real production.
- Search uses PostgreSQL ILIKE rather than Elasticsearch.
- Kubernetes upload storage is a 10Gi ReadWriteOnce volume with required pod affinity co-locating API pods on one node; multi-node operation requires shared or object storage.
- Push tokens are stored, but FCM/APNs delivery is not implemented.
- Core table tenant columns are incomplete; safe empty responses are preferred over unsafe data exposure.

These constraints are deliberate Phase 7B boundaries and must be considered during capacity planning and any multi-replica production deployment.